轨迹规划自动化测试技术方案

一、背景与目标

1.1 测什么

根据topcase进行分析 AWR 轨迹规划Top问题自动化case，针对wbp内部发现的问题>80%，轨迹规划自动化，

只测 **wbp 这一段**：输入一个规划请求包，分层级判断轨迹是否符合预期。不含上游任务管理，不含下游控制。

1.3 目标

1、通路自动化：一条请求能自动发出、自动收结果、自动判对错

2、覆盖工作空间：可达/边界/不可达/奇异分区各有数据，解决奇异点、自碰、不可达，真机有撞机风险

3、存量问题无法回归：研发改一次代码，不知道有没有改坏别的场景；每次代码变更重跑，输出通过率趋势

二、整体架构思路

代码块

```
├─ L0 mock 数据层 / recode录包数据 ——— 造数据（ 关节角数据 and 笛卡尔积数据）
|  ——L0 工作空间判断mock是否可用 / recode播包脚本
├─ L1 请求构造层 ——— YAML → proto，发前自查
├─ L2 发送层 ————— Action client / Service client，超时取消
├─ L3 转换层 ————— wbp 收到 proto → request 对象（我们改不了）
├─ L4 wbp 内部 ——— 四路分支 → 规划器 → IK
├─ L5 结果回程层 —— planner_result → result proto
└─ L6 断言层 ————— 判对错，落盘
```

层级	名称	核心职责
L0	数据层	三种造数据方式：C真机录包(cyber_recorder)、B采样抽取；需判断mock数据是否在工作空间内；播包脚本 play_teleop_traj.sh 基于 cyber_recorder 回放

L1	请求构造层	YAML配置 → proto序列化；发前自查字段完整性和位姿合法性
L2	发送层	Action Client(长时规划) / Service Client(短时IK)；超时取消机制，需区分规划超时和IK失败
L3	转换层	WBP收到proto → request对象，反序列化+字段映射；**黑盒，我们改不了**，只能通过输入输出推断
L4	WBP内部	四路分支(遥操/跟踪/纯IK/规划+IK) → 轨迹规划器 → IK求解；核心验证点：fail_on_ik_failure=true时IK失败必须真报错，不得静默输出坏轨迹
L5	结果回程层	planner_result → result proto；成功返回轨迹点序列，失败返回错误码+原因(IK无解/位置超差/姿态超差/关节超限/超时)
L6	断言层	对比预期vs实际判对错；测试结果+原始数据+日志落盘；成功case校验轨迹合理性，失败case校验错误码准确性

三、系统通路（要测的对象wbp）

3.1 两个入口

入口	通信方式	channel	调用方
串行	Cyber **Action**	arm_planner/generate_joint_trajectory	机器人 C++
并行	Cyber **Service**	arm_planner/wireauto_batch_plan	offline_simulator

这是两个必须分别测的通路，不能只测一个。 BATCH 的并发串号是 ACTION 测不出来的。

3.2 内部四路互斥分支（规划器类型）

`CorePlannerStage.run()` 里四路互斥，每一路都要有 mock 数据：

#	分支	触发条件	mock 要点
1	HOLD	起终点关节差 < 1.0e-3	**必测**
2	line	planner_type == "line"	唯一解 IK 的路径；用 plan_to_frame_waypc curobo 不同
3	curobo 本地	非远程 + curobo	主力；tiny/heavy 两档都要碰
4	cloud	use_remote_planner	128 并行，重点测并发与串号

3.3 请求字段全表（mock 要构造的对象）

planning_requests.zip
4.37MB



plan_20260910_194224_051_success.yaml

3.4 motion_type 七枚举

`LEFT_ARM=1`、`RIGHT_ARM=2`、`BODY_LEFT_ARM=3`、`BODY_RIGHT_ARM=4`、`DUAL_ARM=5`、`WHOLE_BODY=6`、`WHOLE_BODY_WITH_CHASSIS=7`

四、recode录包与播包

五、mock分层

六、断言设计